



Reporte de proyecto Final

Realizado por los estudiantes:

Castillo León Carlos Ramón 21340243

Gámez Oltehua Ángel Manuel 21340251

Gil Cota Jesús Rosario 20340171

Méndez Valenzuela Brandon Ismael 20340140

Carrera: Ingeniería electrónica

Materia: Interfaces y automatización de pruebas.

Maestro: Oscar Indalesio Ramírez Huerta.

Fecha: diciembre 2025.

Índice.

1.- Introducción	4
2.- Descripción del problema.....	5
3.- Objetivo General.....	6
4.- Objetivos Específicos.	6
4.1.-Fundamentación Técnica.	7
5.- Diseño.....	8
5.1 Circuito de giro de motor.	8
5.1.2 Circuito de control de velocidad del motor.	9
5.1.3 integración de ambos circuitos.....	11
5.2- Diseño de la interfaz de usuario (UI) de C#.....	12
5.3.- Diagramas de operación de clases.....	17
5.4- Diseño de base de datos.	20
5.4.1 Tabla <i>TestResult</i>	21
5.4.2 Tabla <i>TestPlan</i>	23
5.4.3. Tabla Modelos.	25
5.4.4. Tabla Productos.	26
5.4.5. Tabla <i>TestStatus</i>	27
6.- Desarrollo.....	28
6.1.- <i>Sealevel</i>	28
6.1.1 Configuración del entorno.	28
6.1.2 Ejemplo de conexión Modbus RTU.	28
6.2 Instrumentos.	33
6.3- Desarrollo De Procedimientos Almacenados.	38
6.3.1- <i>AddTestPlanResult</i>	39

6.3.2- <i>Get_NextTRID.</i>	39
6.3.3- <i>Get_NS_Repeated.</i>	40
6.3.4- <i>Report_TestPlan.</i>	41
6.3.5. <i>SP_Filter.</i>	42
6.3.6- <i>Update_Producto_StatusCode.</i>	44
6.4.- Desarrollo del sistema.	45
6.4.1- Conexión al instrumento.....	45
6.4.2- Prueba del producto.....	46
6.4.3- Búsqueda.	48
7.- Limitaciones.	50
8.-Resultados	50
9.- Conclusión.....	56
10- Bibliografías	56

Fig. 5.1.2- Vista del diseño PCB donde va el circuito de control de velocidad.	10
Fig. 5.1.3- vista del diseño pcb completo.....	11
Fig. 5.2.1. StatusStrip del Form.	12
Fig. 5.2.3- Vista diseño de la cola c#.....	13
Fig. 5.2.3- Visualización de las pruebas en tiempo real c#.....	14
Fig. 5.2.4- Visualización de del historial del Reporte.....	15
Fig. 5.2.5- Vista de la ventana de búsqueda.....	16
Fig. 5.3.1- Conexión de instrumentos.....	17
Fig. 5.3.2- Diagrama de operación para las pruebas de un “producto”.	18
Fig. 5.3.3- Diagrama de operación para la búsqueda.....	19
Fig. 5.4.1- Diagrama de tablas de la base de datos utilizada.....	21



Fig. 8.1 -Resultados de la Register	51
Fig. 8.2- Resultados de queue.	51
Fig. 8.3- Resultados de los test plans.	52
Fig. 8.4- Resultados de los test plans.	52
Fig. 8.5- Resultados de los test plans.	53
Fig. 8.6- Búsqueda filtrada en base a número de serie.....	54
Fig. 8.7- Búsqueda filtrada en base a estatus.	55

1.- Introducción.

El presente documento describe el desarrollo e implementación de un sistema de comunicación y control de instrumentos de laboratorio reales mediante VISA, desarrollado en lenguaje C# bajo la plataforma .NET. El proyecto tiene como objetivo integrar una aplicación de software con equipos físicos como una fuente de poder y un multímetro digital, permitiendo su configuración y lectura de manera automatizada.

El sistema se concibe como una solución orientada a la automatización de pruebas y adquisición de datos, aplicando principios de programación orientada a objetos, uso de interfaces, abstracción de hardware y comunicación mediante comandos SCPI. De esta manera, se busca simular un entorno cercano al utilizado en la industria, donde el control manual de instrumentos es sustituido por sistemas programables y repetibles.

2.- Descripción del problema.

Durante las últimas semanas, la clase de Interfaces de automatización y pruebas conoció de cómo utilizar lo que son inyección de dependencias y los tipos de datos abstractos (ADT) para lo que son el almacenamiento de instrumentos, en donde se podían realizar diferentes operaciones con la intención de familiarizarse con la lógica y beneficios de esta técnica de patrón de diseño dentro de una sencilla interfaz en consola, esto era con el propósito de poder hacer más énfasis en lo que son los conocimientos y principios de inversión de dependencias (SOLID).

Pero ahora se debe utilizar el conocimiento adquirido para un sistema de registro y calibración de instrumentos, en donde la idea es utilizar lo que son diferentes tipos de datos abstractos para la realización de tareas más específicas dentro de una interfaz visual y amigable al usuario en donde este diseñado con fluidez y sea sencillo para cualquiera manejar brindando la información necesaria, acercándose a lo que debe ser un estándar en la industria y simular lo que sería el proceso como si fuese realizándose en físico.

El problema principal radica en que deberíamos considerar al momento de realizar la simulación, ya que existen múltiples consideración y requisitos que debemos cumplir para que la simulación sea lo más cercano a lo que sería un caso en la industria, aparte de que el equipo enfrentara a como la lógica de patrón de diseños debe utilizarse que en este caso son la interpretación de los tipos de datos abstractos y la inyección de dependencias.

Es importante destacar que el circuito a implementar esta interfaz de pruebas queda a interpretación de los estudiantes, por lo cual deben definir las operaciones a realizar para las pruebas.

3.- Objetivo General.

Desarrollar una aplicación con interfaz gráfica (GUI) basada en Windows Forms capaz de comunicarse con instrumentos reales de laboratorio mediante VISA, permitiendo el control de una fuente de poder y la adquisición de mediciones de un multímetro digital de forma automatizada y estructurada.

4.- Objetivos Específicos.

- Diseñar y construir un circuito físico con puntos de medición fijos, de diferentes tipos de señal o magnitud.
- Definir al menos dos modelos distintos, cada uno con parámetros de aceptación diferentes para las pruebas a realizar.
- Permitir la variación controlada de parámetros en el circuito con el fin de generar condiciones de prueba que incluyan casos PASS y FAIL.
- Deberá de registrar los Test plan para cada modelo.
- Registrar previamente los números de serie que se van a probar, indicando el modelo al que pertenece cada uno.
- Desarrollo de una aplicación en C# que empleando librerías de clases y una interface de Windows Forms:
 - o Obtenga el TestPlan correspondiente al modelo de cada número de serie que se probara de la BD.
 - o Controle los instrumentos por GPIB y/o Serial.

- o Realice un muestreo de 30 mediciones por prueba, calcule el promedio con hasta 4 decimales y agregue la unidad de medida.
 - o Evalúe el resultado de la medición contra los límites definidos y determine el estatus PASS/FAIL de cada prueba.
 - o Registre los resultados en la base de datos.
- Implementar la lógica para que, si todas las pruebas de un número de serie son PASS, el producto se marque como PASS; en caso de cualquier falla, se detengan las pruebas y el número de serie se marque como FAIL, pero todas las pruebas realizadas incluyendo la fallada se registren
- Incluir en el sistema un menú de reportes

4.1.-Fundamentación Técnica.

El proyecto se desarrolló bajo las condiciones de las interfaces gráficas de usuario (GUI) en C# con Windows Forms, empleando:

- **Lenguaje:** C# (.NET Framework 8.0)
- **Entorno:** Visual Studio 2026
- **Paradigma:** Programación Orientada a Objetos (POO)
- **Comunicación:** VISA (Ivi.Visa / NationalInstruments.Visa)
- Comunicación con el ssealevel mediante modbus
- **Protocolo:** SCPI (Standard Commands for Programmable Instruments)
- **Tipo de aplicación:** Librería de clases, consola y Windows Forms App.



Se diseñó un formulario principal “Proyecto_Final_Interfaces” con un “TabControl” dividido en cinco secciones operativas: *Register*, *Queue*, *Test panel*, *History* y *search*. Cada módulo cumple con un objetivo funcional específico e independiente, contribuyendo al flujo completo del sistema. El sistema se encuentra desacoplado mediante inyección de dependencias en el cual el servicio (o controlador) depende de interfaces, *I_Instruments*, *I_Source* y *I_Multimeter* son ejemplo de algunas donde en el siguiente capítulo se hablará más a detalle.

5.- Diseño.

5.1 Circuito de giro de motor.

Este circuito utiliza un relé DPDT (*Double pole double throw*) para invertir la polaridad aplicada al motor DC. Al cambiar la polaridad, el sentido del giro se invierte, El accionamiento del relé se realiza mediante un *Sealevel* activado por una interfaz en C# y un diodo 1N4007, el cual protege al circuito contra picos de voltaje generados por la bobina del relé.

Componentes Principales.

- Motor DC 12V.
- Relé DPDT.
- Diodo 1N4007.
- *Sealevel*.
- Fuente de alimentación de 12 volts.
- Fuente de alimentación de 24 volts (para alimentar el *Sealevel*).

Funcionamiento

Al activar un relevador asignado en el *sealevel* en este caso el 2 se activa una sección del motor para que empiece a girar en sentido horario conmutando sus contactos y aplicando una polaridad específica al motor.

Al accionar el relé 3 en el *sealevel* se activa el sentido antihorario, el relé invierte las conexiones internas, cambiando la polaridad del motor.

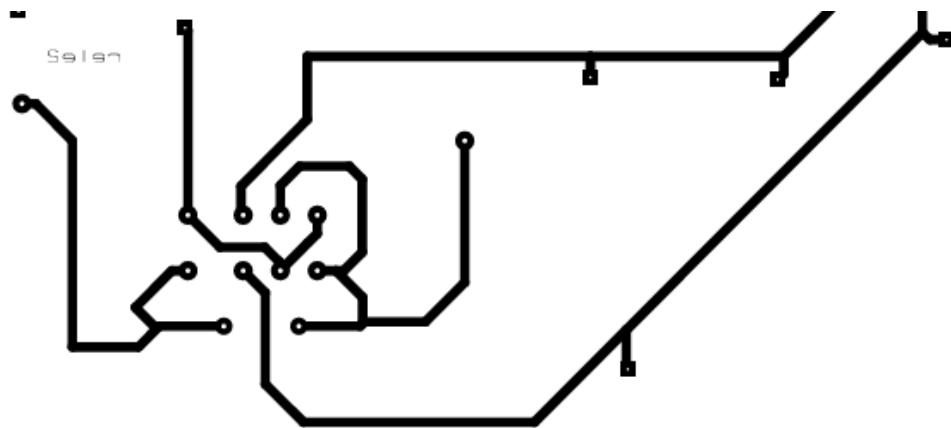


Fig. 5.1- Vista del diseño PCB donde va el circuito de giro.

5.1.2 Circuito de control de velocidad del motor.

El circuito de control de velocidad permite regular las revoluciones del motor DC variando el voltaje efectivo aplicado. Esto se logra mediante un potenciómetro y que actúa como regulador de potencia.

Componentes.

- Potenciómetro de 50k.

- Resistencias.
- Capacitores.
- Diodo de protección.
- Fuente de alimentación de 12 Vdc.

Funcionamiento.

El circuito es activado cuando el *sealevel* activa el relé 1,5,8 separándolo del circuito de giro para evitar perturbaciones en la velocidad, este puede variar su velocidad mediante el potenciómetro variando la corriente y voltaje que llegan al motor.

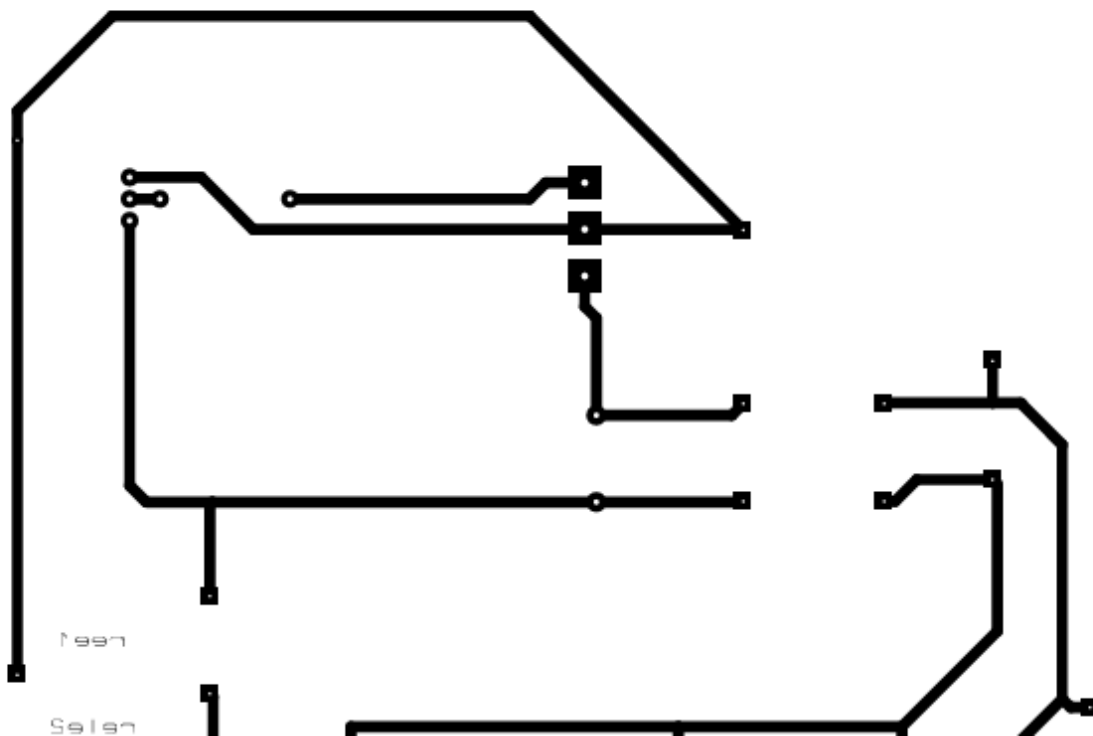


Fig. 5.1.2- Vista del diseño PCB donde va el circuito de control de velocidad.

5.1.3 integración de ambos circuitos.

Al integrar ambos circuitos el control depende completamente del *sealevel*, el cual accionara cada parte del circuito por separado.

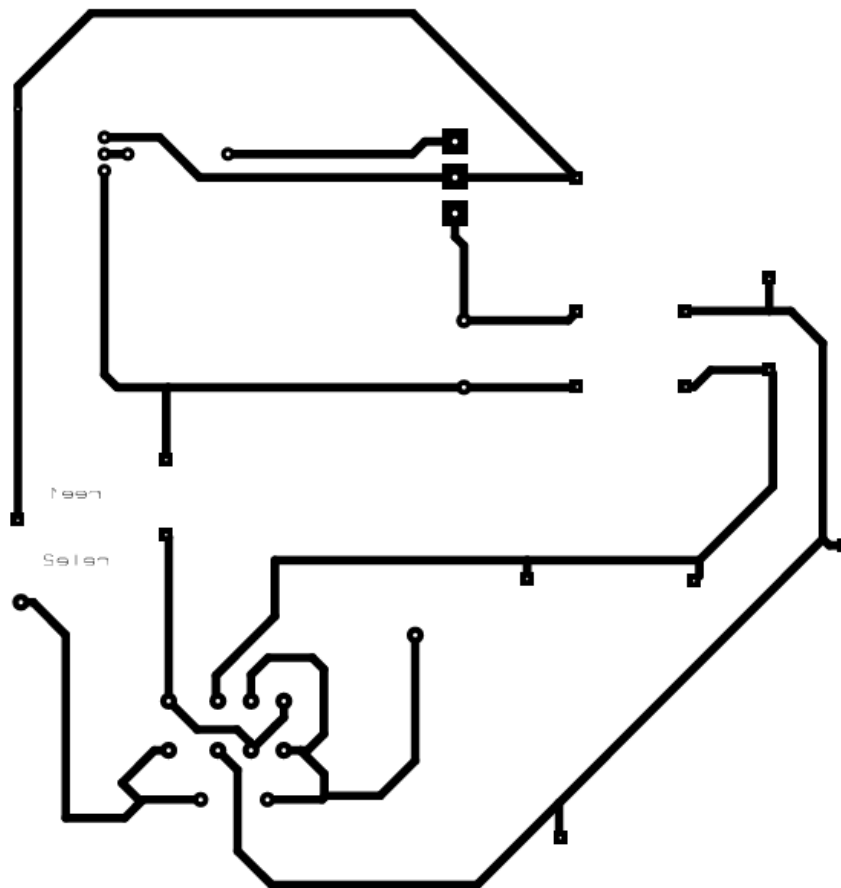
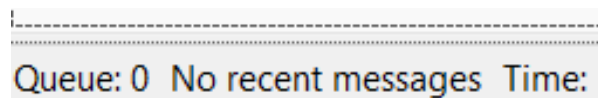


Fig. 5.1.3- vista del diseño pcb completo.

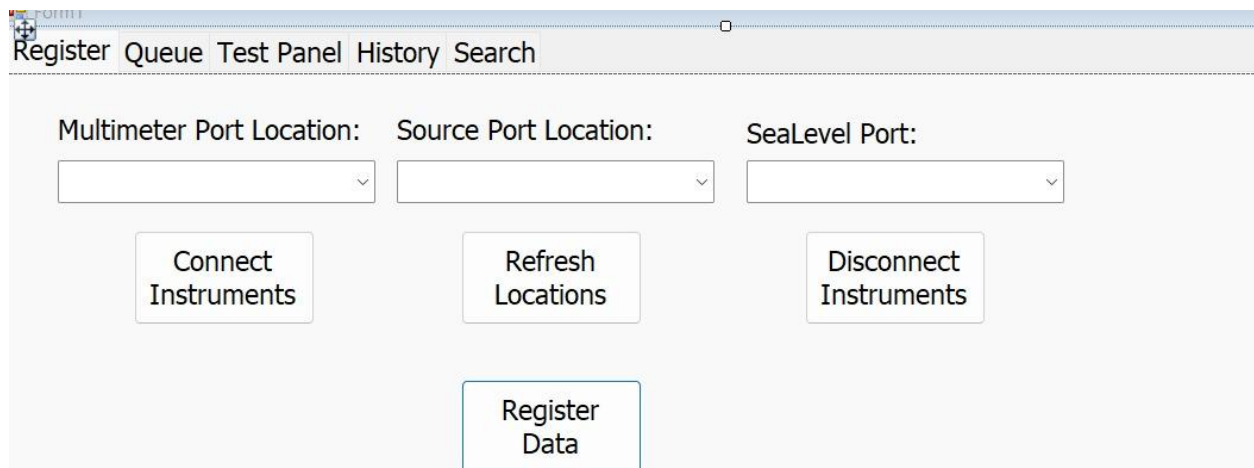
5.2- Diseño de la interfaz de usuario (UI) de C#.

Como se mencionó en el capítulo 4.1, este programa hace uso de 5 ventanas con la intención de ser una interfaz gráfica que conecta con los instrumentos, *Sealevel* y base de datos de SQL para la visualización de datos. El *Form* también está conformado por un *StatusStrip* donde se mostrarán diferentes *Labels* para indicar una referencia a elementos cruciales del programa, estos siendo la cola (FIFO), notificaciones o mensajes recientes y el tiempo actual. La siguiente imagen indica los elementos mencionados. Es importante destacar que no importa a que ventana el usuario decide observar, el *StatusStrip* siempre se encontrara visible.



Queue: 0 No recent messages Time:

Fig. 5.2.1. *StatusStrip* del *Form*.



The screenshot shows a Windows application window titled "FORM1". The menu bar includes "Register", "Queue", "Test Panel", "History", and "Search". The main area contains three dropdown menus labeled "Multimeter Port Location:", "Source Port Location:", and "SeaLevel Port:". Below these are three buttons: "Connect Instruments", "Refresh Locations", and "Disconnect Instruments". At the bottom center is a button labeled "Register Data".

Fig. 5.2.2- Vista diseño de registrar datos.

Como se muestra en la figura 5.2.2 que corresponde a ventana de registro de instrumentos del sistema o *Register*. En el apartado el usuario puede seleccionar los puertos de comunicación para cada equipo de medición, desde la misma visa se controlan las acciones principales de conexión, permitiendo conectar los instrumentos, actualizar o refrescar los puertos disponibles y desconectar los equipos. El botón de registro debe cargar todos los *TestPlan* a evaluar por los instrumentos en donde se podrán observar adelante.

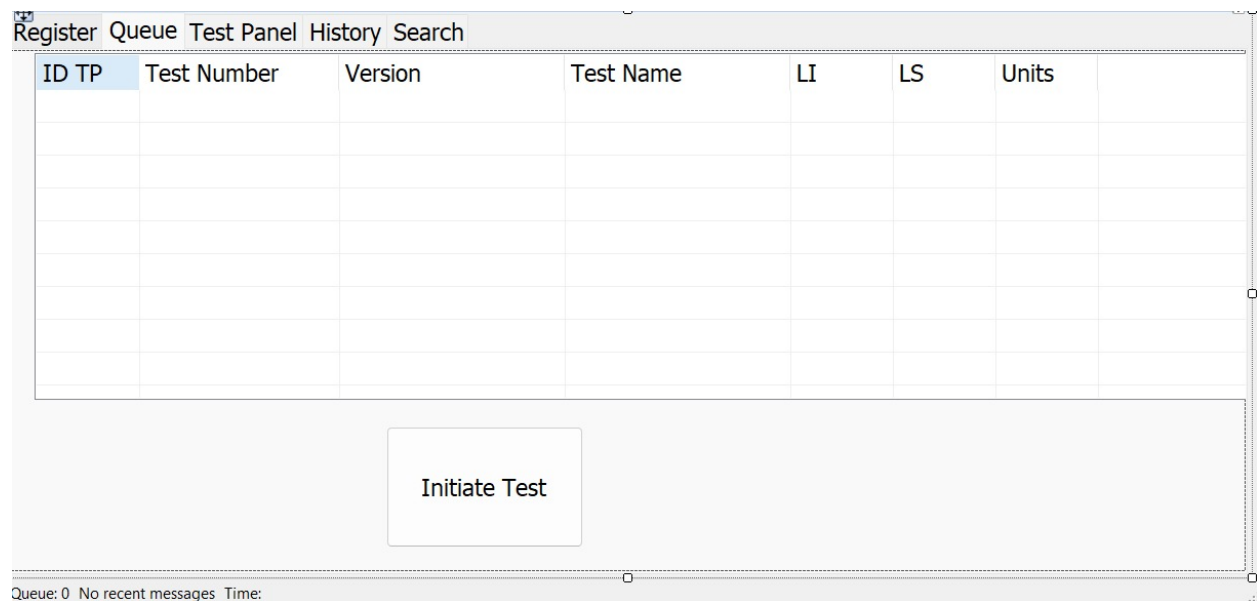


Fig. 5.2.3- Vista diseño de la cola c#.

La figura muestra la interfaz del módulo de selección e inicio de pruebas dentro de la aplicación, el área principal de la interfaz está conformada por una tabla de datos (el cual es un objeto *ListView*), en la cual se listan los planes o pruebas disponibles. La tabla incluye campos como *ID TP*, *Test Number*, *Version*, *Test Name*, *LI*, *LS* y *Units*, los cuales permiten identificar cada prueba, su versión y los límites inferior y superior de operación, así como las unidades de medición correspondientes.

En la parte inferior se encuentra el botón *Initiate Test*, cuya función es iniciar la prueba seleccionada dentro del *ListView*, esa prueba está diseñada para realizar las operaciones de todos los *TestPlan* del producto registrado.

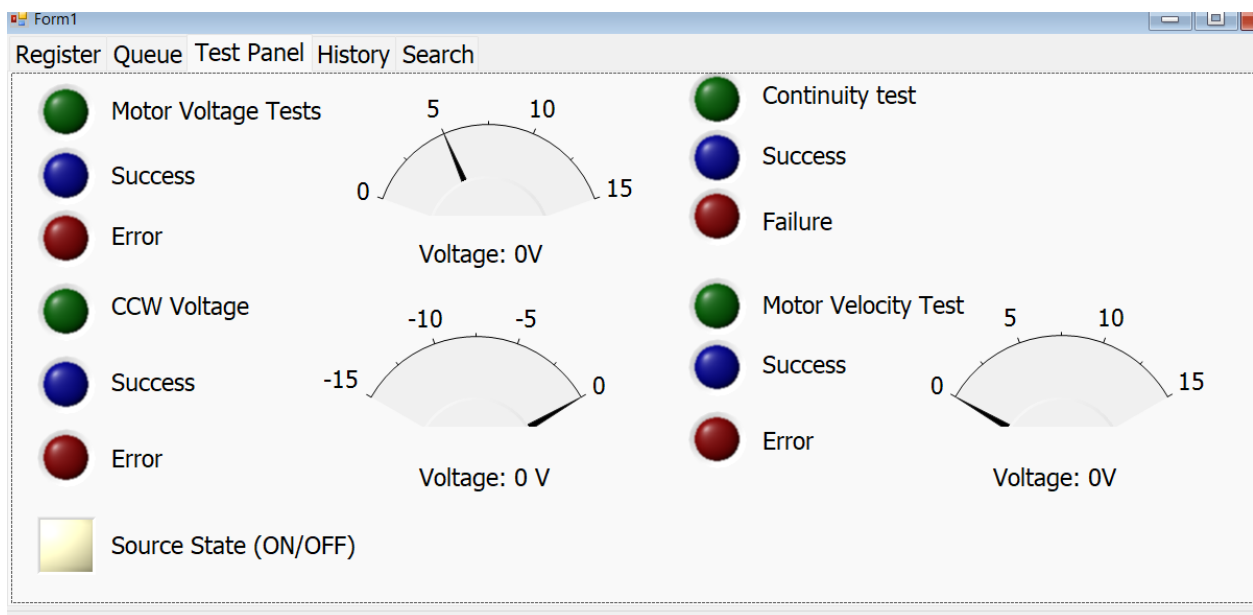


Fig. 5.2.3- Visualización de las pruebas en tiempo real c#.

La figura muestra la interfaz gráfica de un sistema de pruebas eléctricas utilizado para la medición de voltaje en un motor y la verificación de continuidad eléctrica, se emplean indicadores visuales para mostrar el estado de las pruebas, donde el LED de color verde indica prueba activa, el azul éxito y el rojo error. En la parte central se presentan tres mediciones analógicas (*meter*): uno para el voltaje del motor en sentido normal (0 a 15V), para el voltaje en sentido contrario (*CCW*) con escala negativa y el ultimo es para una prueba de velocidad diferente a la primera prueba (con la misma escala de 0 a 15V). Ambos medidores indican 0V, lo que señala que el motor no está energizado al momento de inicializar la aplicación y también cuando se desea realizar una nueva prueba para un producto de diferente número de serie. La sección de prueba de continuidad es

similar a la de los voltajes, pero a diferencia de no hay escala (meter) para lectura analógica, as que permite verificar la continuidad del circuito, utilizando igualmente indicadores de estado por colores. Finalmente, el LED del estado de la fuente muestra su estado.

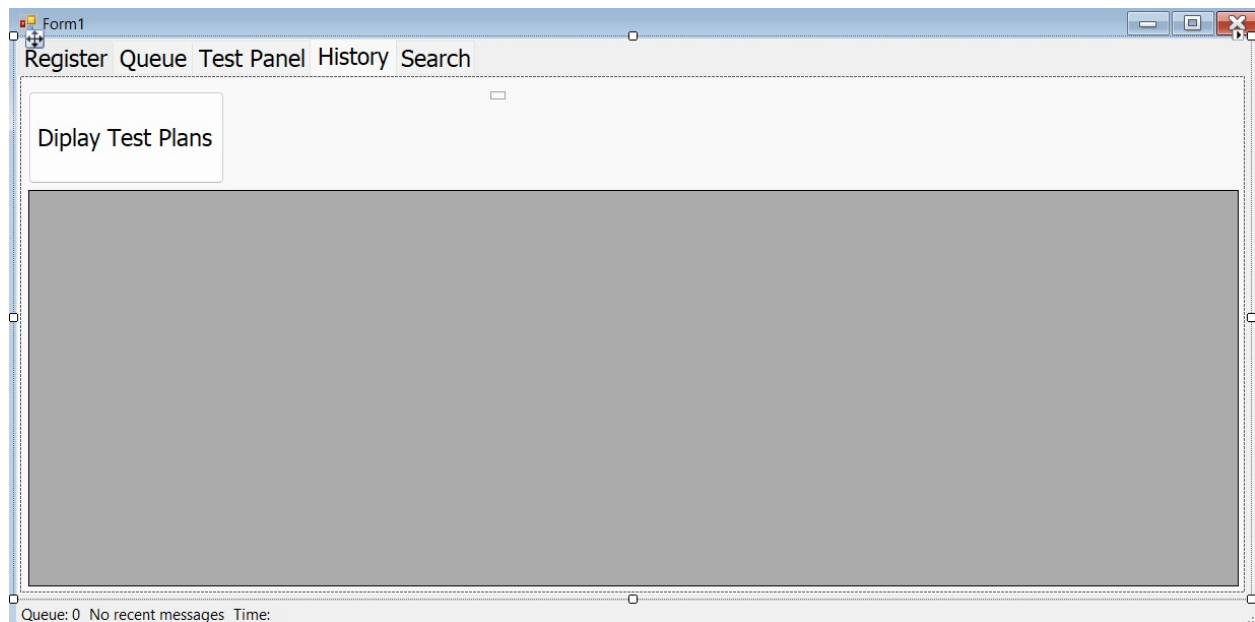
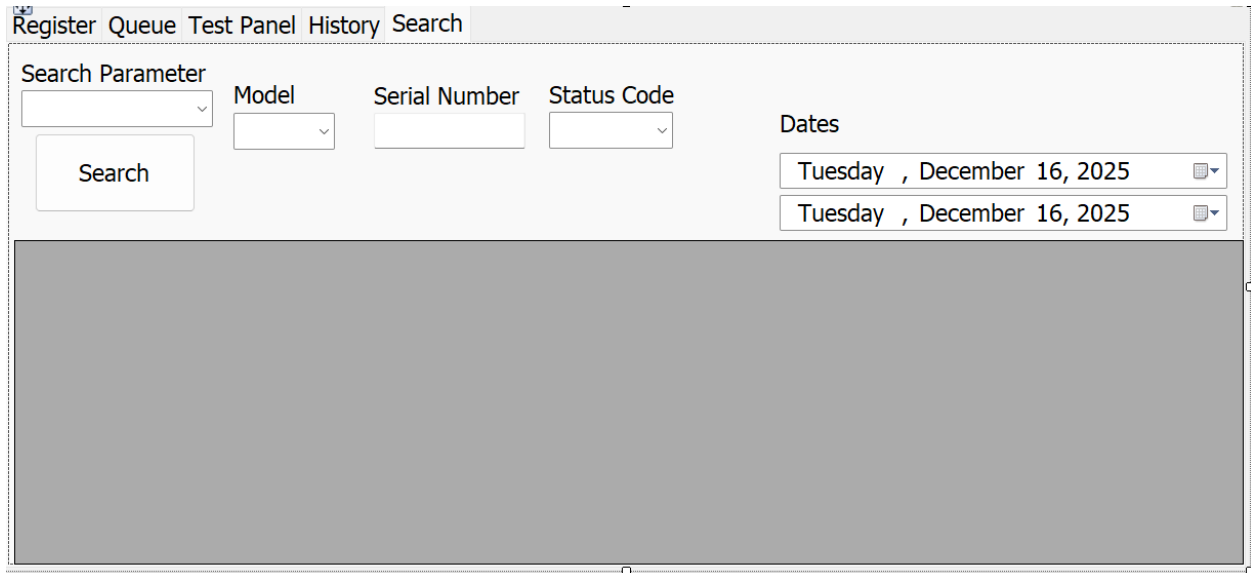


Fig. 5.2.4- Visualización de del historial del Reporte.

La figura 5.2.4 presenta los resultados de las pruebas que se involucran en el proyecto las cuales permiten consultar el historial. El botón (*Display Test Plans*), destinado a mostrar o cargar los *TestResults*. La mayor parte de la interfaz está ocupada por un área central de trabajo, destinada a la visualización de información utilizando un DGV (*DataGridView*).



The screenshot shows a software window titled "Register Queue Test Panel History Search". The "Search" tab is active. The interface includes a "Search Parameter" dropdown menu, a "Model" dropdown menu, a "Serial Number" text input field, and a "Status Code" dropdown menu. A "Search" button is located below the "Search Parameter" dropdown. To the right, under the "Dates" label, there are two date pickers, both showing "Tuesday , December 16, 2025". The main area of the window is a large, empty gray rectangle, likely intended for displaying search results.

Fig. 5.2.5- Vista de la ventana de búsqueda.

En este apartado se muestran los componentes para iniciar la búsqueda de un elemento en específico, para iniciar todos los *ComboBox* presentes contendrán todos los posibles elementos que se puedan encontrar en el campo respectivo, por ejemplo; modelo contendría “A” o “B” que son las 2 diferentes versiones del producto. El *ComboBox* de *Search Parameter* indicará cuál de los diferentes campos presentes se tomará en cuenta para filtrar los elementos no deseados.

Los únicos 2 elementos diferentes para las consultas son el *TextBox* del número de serie, con la intención de que el operador sea capaz de consultar a su voluntad y los 2 *DateTimePicker* para consultar el rango de fechas a buscar.

5.3.- Diagramas de operación y clases.

En adelante se hablarán sobre la arquitectura de programa, lo que debe cumplir y como será la lógica para el desarrollo de este proyecto.

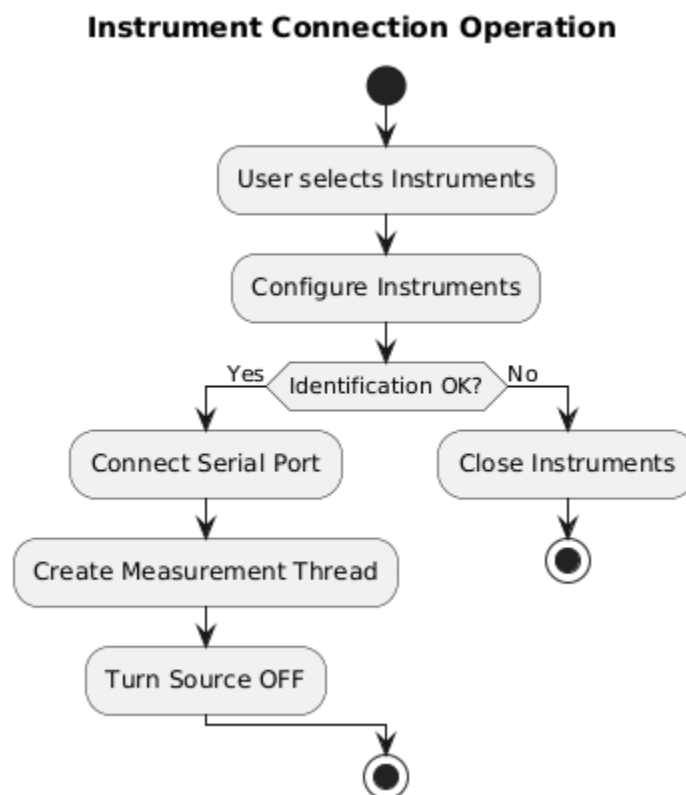


Fig. 5.3.1- Conexión de instrumentos.

En este diagrama muestra un sistema que, al momento de conectar los instrumentos, el programa pide identificarlos para que, al momento de conectarse, asegurarse que sean los adecuados de manera autónoma.

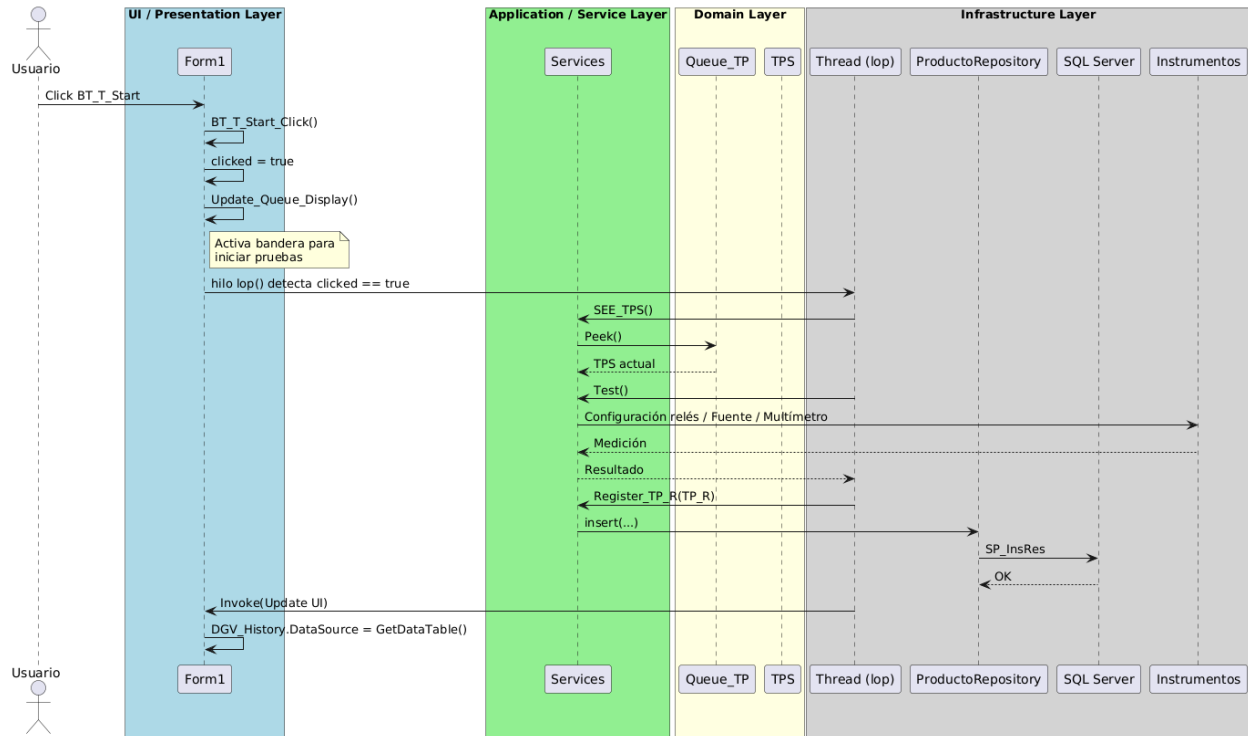


Fig. 5.3.2- Diagrama de operación para las pruebas de un “producto”.

Esta es la sección más importante del programa, permitiendo comunicarse con diferentes partes del programa para realizar las pruebas y evaluarlas donde se podrán ver los resultados en la sección *Test Panel* de la interfaz para luego ir registrándolos en la base de datos.

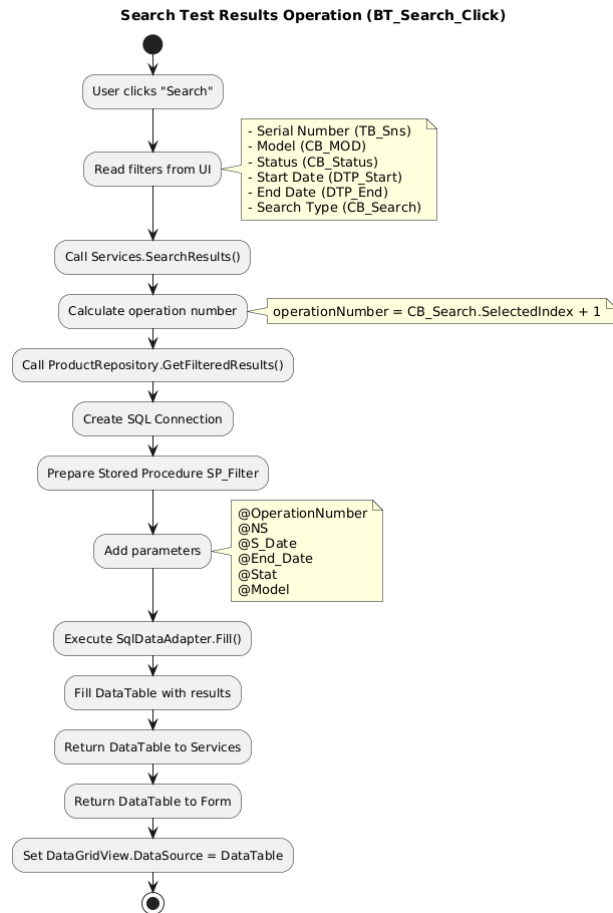


Fig. 5.3.3- Diagrama de operación para la búsqueda.

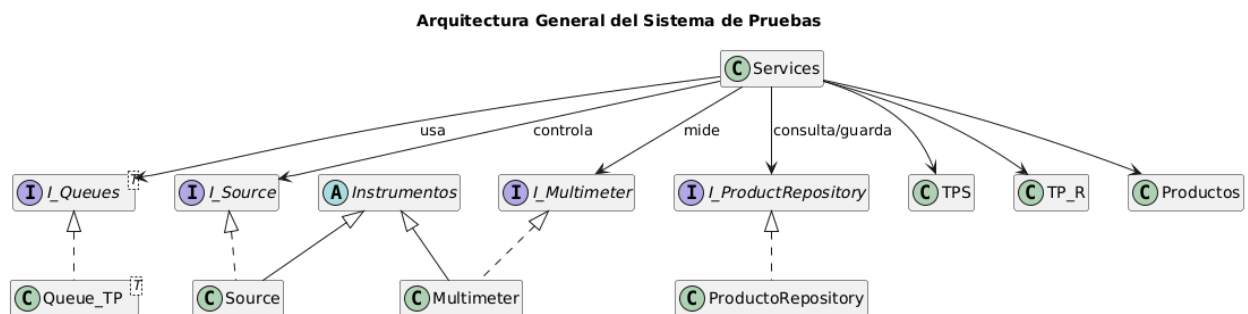


Fig. 5.3.4- Diagrama de clase simplificado

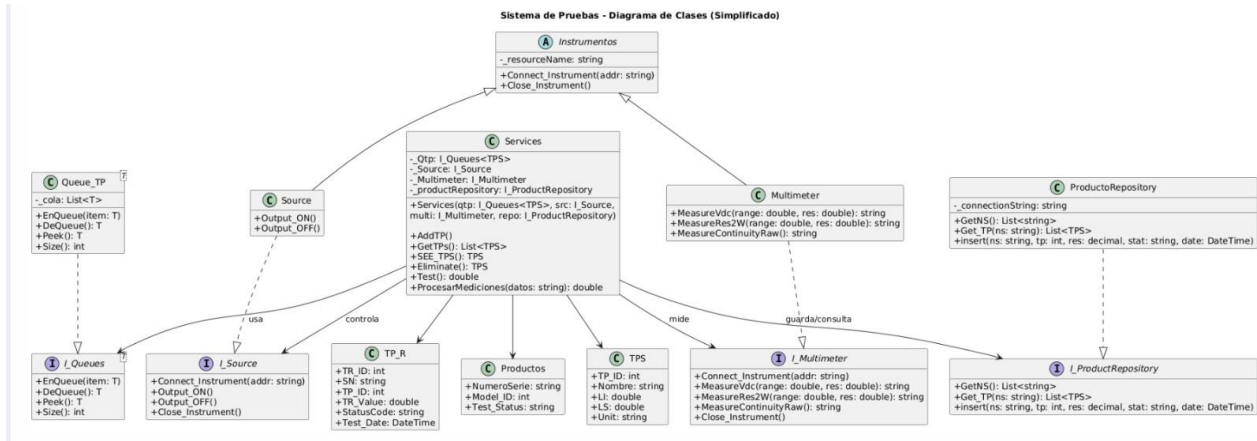


Fig. 5.3.5- Diagrama de clase desarrollado

En la Figura 5.3.5 el diseño del sistema está basado en una estructura modular, utilizando clases abstractas, interfaces y clases concretas para asegurar flexibilidad, escalabilidad y facilidad de mantenimiento.

Los instrumentos encapsulan los detalles de conexión y operación, la cola de pruebas protege su estructura interna, las clases de dominio resguardan la información de pruebas, resultados y productos, y el repositorio oculta la lógica de acceso a la base de datos.

5.4- Diseño de base de datos.

La base de datos fue proporcionada por el maestro con la intención de dar una fundación para que los estudiantes puedan enfocarse en desarrollar la programación de la interfaz, controlador, y demás sistemas necesarios para el desarrollo del proyecto. Adelante se explicará la estructura de la base de datos.

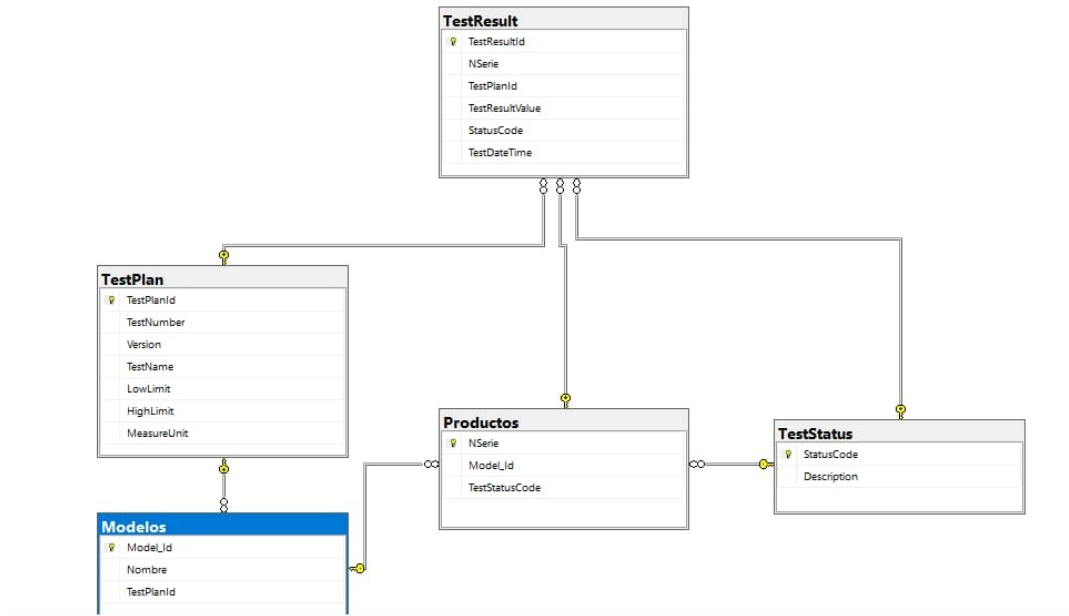


Fig. 5.4.1- Diagrama de tablas de la base de datos utilizada.

5.4.1 Tabla *TestResult*.

❖ *TestResultId*.

- Para qué sirve: Identifica de forma única cada resultado de prueba.
- Dónde se usa:
 - Generado automáticamente al insertar un resultado.
 - Consultado en *Get_NextTRID* para referencia del siguiente registro.
 - Utilizado en reportes y consultas históricas.

❖ *NSerie*.

- Para qué sirve: Relaciona el resultado con un producto específico.
- Dónde se usa:

- Inserción de resultados de prueba.
- Consultas en *TestResult_History*.
- Relación directa con la tabla Productos.

❖ *TestPlanId*.

- Para qué sirve: Indica qué plan de prueba se aplicó.
- Dónde se usa:
 - Inserción de resultados.
 - Reportes en *Report_TestPlan*.
 - Relación con la tabla *TestPlan*.

❖ *TestResultValue*.

- Para qué sirve: Almacena el valor medido durante la prueba.
- Dónde se usa:
 - Registro de pruebas.
 - Visualización en historial y reportes.
 - Evaluación contra límites definidos.

❖ *StatusCode*.

- Para qué sirve: Define si la prueba fue aprobada o rechazada.
- Dónde se usa:
 - Inserción de resultados.

- Relación con *TestStatus*.
- Consultas en *TestResult_History*.

❖ *TestDateTime*.

- Para qué sirve: Registra cuándo se ejecutó la prueba.
- Dónde se usa:
 - Ordenamiento de resultados en *TestResult_History*.
 - Reportes cronológicos.

5.4.2 Tabla *TestPlan*.

❖ *TestPlanId*

- Para qué sirve: Identificador único del plan de prueba.
- Dónde se usa:
 - Relación con Modelos.
 - Inserción y consulta de resultados (*TestResult*).
 - SP: *AddTestPlan*, *UpdateTestPlan*, *Report_TestPlan*.

❖ *TestNumber*

- Para qué sirve: Número que identifica la prueba.
- Dónde se usa:
 - Definición del plan.
 - Visualización en reportes (*Report_TestPlan*).

❖ *Version*

- Para qué sirve: Control de versiones del plan de prueba.
- Dónde se usa:
 - Actualización en *UpdateTestPlan*.
 - Reportes de pruebas.

❖ *TestName*.

- Para qué sirve: Describe el tipo de prueba realizada.
- Dónde se usa:
 - Reportes.
 - Historial de pruebas.
 - Administración de planes.

❖ *LowLimit / HighLimit*.

- Para qué sirve: Define los rangos aceptables del resultado.
- Dónde se usa:
 - Evaluación lógica de resultados.
 - Reportes comparativos.

❖ *MeasureUnit*.

- Para qué sirve: Indica la unidad de medición.
- Dónde se usa:

- Interpretación del resultado.
- Reportes (*Report_TestPlan*).

5.4.3. Tabla Modelos.

❖ *Model_Id.*

- Para qué sirve: Identifica el modelo del producto.
- Dónde se usa:
 - Relación con Productos.
 - Actualización y alta (*AddModel*, *UpdateModelo*).

❖ *Nombre.*

- Para qué sirve: Nombre técnico del modelo.
- Dónde se usa:
 - Reportes (*TestResult_History*).
 - Visualización en la aplicación.

❖ *TestPlanId.*

- Para qué sirve: Define qué pruebas corresponden al modelo.
- Dónde se usa:
 - Al registrar productos.
 - Relación automática con resultados.

5.4.4. Tabla Productos.

❖ *NSerie.*

- Para qué sirve: Identificador único del producto.
- Dónde se usa:
 - Registro de resultados (TestResult).
 - Validación (Get_NS_Repeated).
 - Historial y reportes.

❖ *Model_Id.*

- Para qué sirve: Indica a qué modelo pertenece el producto.
- Dónde se usa:
 - Consulta del plan de prueba aplicable.
 - Relación con Modelos.

❖ *TestStatusCode*

- Para qué sirve: Resume el estado final del producto.
- Dónde se usa:
 - Debe actualizarse al terminar la prueba para un modelo.
 - Consultas rápidas de estado.

5.4.5. Tabla *TestStatus*.

❖ *StatusCode*.

- Para qué sirve: Código del estado de prueba.
- Dónde se usa:
 - En *TestResult*.
 - En Productos.
 - Relación con resultados y estado final.

❖ *Description*.

- Para qué sirve: Explicación del estado.
- Dónde se usa:
 - Visualización para el usuario en esta misma tabla.

6.- Desarrollo.

En esta sección se describirán todas las consideraciones o trabajos realizados durante la realización de este programa con la intención de explicar porque o cómo fue que realizamos el proyecto de tal manera para cumplir con los objetivos establecidos en el inicio del documento.

6.1.- *Sealevel*.

Los dispositivos *Sealevel* permiten controlar entradas y salidas digitales y analógicas mediante protocolos estándar como Modbus. Con Modbus4 para C#, se permite enviar comandos de lectura/escritura a registros del *Sealevel* Modbus RTU (Puerto serial) o Modbus TCP.

6.1.1 Configuración del entorno.

- Instalar Modbus4: *Dotnet add package Modbus4*.
- Configurar el puerto serial.
- Tasa de baudios: según especificaciones del *SeaLevel*.
- Paridad: *None* o *Even* según el modelo
- Utilizar una librería que sea capaz de distinguir el puerto del *SeaLevel*, en este caso *System.IO.Ports*.

6.1.2 Ejemplo de conexión Modbus RTU.

Aquí se pueden presenciar fragmentos esenciales para el uso del *SeaLevel*. En el siguiente fragmento se puede apreciar las librerías utilizadas.

```
using System;  
using System.IO.Ports;  
using Modbus.Device;
```

Este fragmento demuestra todos los elementos esenciales al crear la configuración del puerto serial.

```
// Configuración del puerto serial  
SerialPort puerto = new SerialPort(  
    "COM3",      // Puerto COM  
    9600,        // Velocidad en baudios  
    Parity.None, // Sin paridad  
    8,           // Bits de datos  
    StopBits.One // Bit de parada  
);
```

```
// Apertura del puerto  
puerto.Open();  
  
// Creación del maestro Modbus RTU  
var master = ModbusSerialMaster.CreateRtu(puerto);  
  
// ID del esclavo Modbus  
byte slaveID = 1;  
  
// Lectura de una entrada digital (Input 0)  
bool [] inputs = master.ReadInputs(slaveID, 0, 1);  
Console.WriteLine("Estado de la entrada 0: " + inputs [0]);  
  
// Escritura de una salida digital (Coil 5)
```



```
master.WriteSingleCoil(slaveID, 5, true);  
Console.WriteLine("Salida 5 activada");  
  
// Cierre del puerto serial  
puerto.Close();  
}  
}
```

Este es un ejemplo básico de las diferentes operaciones posibles utilizando la librería de modbus.

```
// FUNCIONES PARA RELÉS  
  
private void ActivarRele(int número)  
{  
    int real = numero - 1;  
    master.WriteSingleCoil(slaveId, (ushort)real, true);  
}  
  
private void DesactivarRele(int número)  
{  
    int real = numero - 1;  
    master.WriteSingleCoil(slaveId, (ushort)real, false);  
}
```



Las funciones anteriores son para activar el relé o desactivar, este cuenta con un restador dado que el programa lo mira del 1 al 8, pero el sealevel lo percibe desde el 0 al 7 como bist.

```
pruebaVelocidadActiva = true;

bt_velocidad.Enabled = false;
bt_giro.Enabled = false;

ActivarRele(5);
ActivarRele(8);
ActivarRele(1);

await Task. Delay (10000); // 10 s

DesactivarRele(1);
DesactivarRele(5);
DesactivarRele(8);

pruebaVelocidadActiva = false;

bt_velocidad.Enabled = true;
bt_giro.Enabled = true;
```

Secuencia de activación del circuito para la prueba de velocidad, cierra los relevadores 1,5,8 para poder separarlo del circuito de giro, quitándole la energía y activando solo el de velocidad el cual es variado mediante un potenciómetro simulando distintos modelos.



```
pruebaGiroActiva = true;

bt_velocidad.Enabled = false;
bt_giro.Enabled = false;

ActivarRele(2);
await Task.Delay(7000);
DesactivarRele(2);

// ---- Espera 1 s ----
await Task.Delay(1000);

ActivarRele(3);
await Task.Delay(9000);
DesactivarRele(3);

pruebaGiroActiva = false;

bt_velocidad.Enabled = true;
bt_giro.Enabled = true;
```

Secuencia de giro del motor siendo controlado con el sealevel, activando el giro en sentido horario con el relé 2 y en sentido antihorario con el relé 3.

6.2 Instrumentos.

Esta parte del código, definiremos la capa de servicio encargada de comunicar una aplicación en C# con instrumentos reales de laboratorio (fuente de poder y multímetro) utilizando VISA (Ivi.Visa / NationalInstruments.Visa) y una jerarquía de clases orientada a interfaces (I_Instruments, I_Source, I_Multimeter). Su propósito principal es ofrecer una abstracción única y reutilizable para conectar, configurar y leer instrumentos físicos desde el software, evitando que el resto del proyecto tenga que preocuparse por detalles de bajo nivel como sesiones VISA, timeouts o manejo manual de comandos SCPI.

```
namespace Service
{
    public abstract class Instrumentos : I_Instruments, IDisposable
    {
        private string _resourceName;
        private ResourceManager rm;
        protected MessageBasedSession mbSession;
```

La clase abstracta Instrumentos actúa como la base para todos los dispositivos VISA. Centraliza la conexión, la comunicación SCPI y la liberación de recursos. Su propósito es evitar código duplicado y ofrecer una estructura uniforme para cualquier equipo del laboratorio, manteniendo un diseño profesional y escalable.

```
public void Close_Instrument()
{
    if (mbSession != null)
```

```
{  
    mbSession.Dispose();  
    mbSession = null;  
}  
  
if (rm != null)  
{  
    rm.Dispose();  
    rm = null;  
}  
}
```

Método encargado de cerrar la conexión y liberar los recursos VISA ocupados. Garantiza que la sesión no quede abierta y que no existan bloqueos de comunicación. Este proceso es crítico en aplicaciones de medición automatizada.

```
public void Dispose()  
{  
    Close_Instrument();  
}
```

Implementación del patrón IDisposable para permitir un cierre seguro del instrumento.

```
public void Connect_Instrument(string resourceName)  
{  
    try
```

```
{  
    rm = new ResourceManager();  
    mbSession = (MessageBasedSession)rm.Open(resourceName);  
    mbSession.ReadStatusByte();  
    mbSession.TimeoutMilliseconds = 5000;  
}  
catch (Exception ex)  
{Console.WriteLine("Error connecting to instrument: " + ex.Message); }  
}
```

Este es un método que establece la conexión con el instrumento mediante su *ResourceName*. Crea el *ResourceManager*, abre la sesión y configura el *timeout*. Este método es la base para iniciar cualquier proceso de medición.

```
public void Write(string command)  
{  
    if (mbSession != null)  
    {  
        mbSession.RawIO.Write(command);  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

Envía un comando SCPI al instrumento. Es la operación fundamental de control, permitiendo cambiar configuraciones o activar funciones internas del equipo.

```
public string Read()  
{  
    if (mbSession != null)  
    {
```

```
        return mbSession.RawIO.ReadString(50000);  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

Realiza una lectura desde el instrumento después de un comando de consulta. Se usa para obtener mediciones o respuestas de estado del equipo físico.

```
public string Query(string command)  
{  
    if (mbSession != null)  
    {  
        Write(command);  
        return Read();  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

Ejecuta una consulta completa: envía un comando SCPI y lee su respuesta. Este método simplifica la interacción con funciones tipo "?": *IDN?, READ?, MEAS?

```
public string[] ListResources()
```

```
{  
    using (var resourceManager = new ResourceManager())  
    {  
        var resources = resourceManager.Find("?*INSTR");  
        return resources.ToArray();  
    }  
}
```

Escanea el sistema en búsqueda de instrumentos disponibles mediante VISA. Permite detectar automáticamente qué equipos están conectados al PC.

```
public string Identify()  
{  
    return Query("*IDN?");  
}
```

Devuelve la identificación del instrumento mediante el comando estándar *IDN? es el primer paso para confirmar comunicación correcta y validar el modelo del equipo.

```
public class Source : Instrumentos, I_Source  
{  
    public void Output_OFF()  
    {  
        Write("OUTP OFF");  
    }  
}
```

La clase Source representa una fuente de poder programable controlada mediante SCPI. Permite encender/apagar la salida y ajustar voltaje/corriente de forma precisa. Está diseñada para pruebas automáticas donde se requiere control eléctrico exacto.

```
public class Multimeter : Instrumentos, I_Multimeter
{
    public string MeasureIdc(double range, double resolution)
    {
        Write($"CONF:CURR:DC {range},{resolution}");
        Write("SAMPLE:COUNT 30");
        return Query("READ?");
    }
}
```

La clase Multimeter representa un multímetro digital programable. Implementa funciones esenciales de medición automatizada: corriente DC, resistencia, voltaje DC y continuidad, todas utilizando comandos SCPI. Se usa en rutinas de caracterización, pruebas de laboratorio y registros de datos.

Antes de desarrollar los procedimientos almacenados, se realizó la creación y definición de la base de datos, en la que se basó la estructura del proyecto, que proporcionó el maestro.

Con la base de datos previamente analizada y adaptada a los requerimientos del sistema, se procedió al desarrollo de los procedimientos almacenados, los cuales fueron diseñados directamente sobre esta estructura.

También, cabe destacar que, para probar la funcionalidad de los procedimientos almacenados, fue creado un programa en C# para ejecutar los comandos (*Query*) directamente de ahí y que extraiga información de la base de datos e inserte información a la base de datos.

6.3- Desarrollo De Procedimientos Almacenados.

En este apartado se hablaron de los SP más relevantes durante el desarrollo del proyecto.

6.3.1- *AddTestPlanResult*.

Estos procedimientos forman el grupo encargado de la creación de elementos principales dentro del sistema. *AddTestPlan* se enfoca en registrar los resultados de un *TestPlan* probado, recibiendo parámetros como el número asignado a la prueba, la versión entre otros con la finalidad de agregarlos al historial.

AddTestPlanResult
<pre> USE [DB_reference222] GO SET ANSI_NULLS ON GO SET QUOTED_IDENTIFIER ON GO ALTER PROCEDURE [dbo].[SP_InsRes] @NS VARCHAR(50), @TP_ID INT, @res DECIMAL (18,6), @status VARCHAR (20), @DateTime DATETIME AS BEGIN SET NOCOUNT ON; INSERT INTO TestResult (NSerie, TestPlanId, TestResultValue, StatusCode, TestDateTime) VALUES (@@NS, @TP_ID, @res, @status, @DateTime); END </pre>

6.3.2- *Get_NextTRID*.

Get_NextTRID se encarga de obtener el próximo *TestResultId* disponible en la tabla *TestResult*, asegurando que cada resultado de prueba tenga una identificación única, comparten la misma

lógica basada en la lectura del valor máximo existente y la generación del siguiente número consecutivo.

Get_NextTRID
<pre> ALTER PROCEDURE [dbo].[Get_NextTRID] -- Add the parameters for the stored procedure here AS BEGIN -- SET NOCOUNT ON added to prevent extra result sets from -- interfering with SELECT statements. SET NOCOUNT ON; -- Insert statements for procedure here SELECT MAX(TR.TestResultId)+1 AS NEXT FROM TestResult AS TR END </pre>

6.3.3- *Get_NS_Repeated.*

Su función es determinar si un número de serie ya existe dentro de la tabla Productos. Para ello realiza una búsqueda directa y devuelve un resultado categórico: “REPETIDO” cuando encuentra coincidencias y “NO REPETIDO” cuando no existen registros previos, para impedir que se registren productos duplicados y asegurar que cada número de serie represente un único dispositivo dentro del proceso.

<pre> ALTER PROCEDURE [dbo].[Get_NS_Repeated] -- Add the parameters for the stored procedure here @NSerie nvarchar (100) AS BEGIN -- SET NOCOUNT ON added to prevent extra result sets from -- interfering with SELECT statements. SET NOCOUNT ON; </pre>

```
-- Insert statements for procedure here
if EXISTS (select 1 from Productos where NSerie = @NSerie)
begin
    select 'REPETIDO' as Status;
end
else
begin
    select 'NO REPETIDO' as Status;
end
END
```

6.3.4- *Report_TestPlan.*

Su función es unir la información proveniente de tablas como TestPlan, TestResult, TestStatus y Productos para ver el comportamiento del producto dentro del proceso de prueba.

```
ALTER PROCEDURE [dbo].[Report_TestPlan]
-- Add the parameters for the stored procedure here
AS
BEGIN
-- SET NOCOUNT ON added to prevent extra result sets from
-- interfering with SELECT statements.
SET NOCOUNT ON;

-- Insert statements for procedure here
select TP.TestPlanId, TP.TestName, TP.TestNumber, TP.Version, TP.LowLimit, TP.HighLimit, TP.MeasureUnit,
TR.NSerie, TR.TestResultValue, TR.TestDateTime, TR.StatusCode,
TS.Description as StatusDescription from TestPlan as TP
left join TestResult as TR on TP.TestPlanId = TR.TestPlanId
left join TestStatus as TS on TR.StatusCode = TS.StatusCode
```

```
order by TP.TestPlanId, TR.TestDateTime  
END
```

6.3.5. *SP_Filter*.

Este se encarga de poder filtrar a partir de los mismos parámetros que se mostraban en la *Fig. 5.2.5.* para obtener solo los elementos deseados y luego pasarlos a el programa de *Forms App*.

Originalmente se planeaba realizar 4 diferentes procedimientos almacenados, pero después de una investigación se logró realizar dentro de un solo, simplificando la cantidad de código en utilizar tanto en SQL y en C# (dado a que para cada SP se requiere un método para el *productRepository*).

```
ALTER PROCEDURE [dbo].[SP_Filter]  
  
    @OperationNumber int,  
    @NS VARCHAR(50),  
    @S_Date Date ,  
    @End_Date Date ,  
    @Stat varChar(20),  
    @model varChar(10)  
  
AS  
BEGIN  
  
    ----- Búsqueda de Numero de Serie  
  
    if @OperationNumber = 1  
    Begin  
        Select * From TestResult as r  
        where r.NSerie like '%' + @NS + '%'  
    End  
  
    ----- Búsqueda por modelo  
  
    if @OperationNumber = 2  
    Begin
```

```
if @model = 'A'
    begin
        SELECT * FROM TestResult as r
        where r.TestPlanId >= 1
        And r.TestPlanId <=4
    End
if @model = 'B'
    Begin
        SELECT * FROM TestResult as r
        where r.TestPlanId >= 5
        And r.TestPlanId <=8
    End
End

----- Búsqueda de fecha
iF @OperationNumber = 3
Begin
    SELECT * FROM TestResult as r
    where r.TestDateTime >= @S_Date
    And r.TestDateTime < DATEADD(Day, 1, @End_Date) --
End

-- Checa para los diferentes valores de status
if @OperationNumber = 4
Begin
    Select * From TestResult
    Where StatusCode = @Stat
End
```

END

6.3.6- *Update_Producto_StatusCode.*

Este grupo integra todos los procedimientos encargados de realizar modificaciones sobre registros existentes. UpdateModelo permite ajustar información asociada a un modelo registrado, como su nombre o el TestPlanId asignado. Update_Producto_StatusCode actualiza el estado de un producto específico mediante su número de serie, reflejando si se encuentra aprobado o rechazado, por último, UpdateTestPlan permite modificar parámetros de un plan de prueba ya existente, como el número del test, la descripción, los límites o la unidad, comparten la finalidad de mantener los datos actualizados sin necesidad de eliminar o replicar registros.

Update_Producto_StatusCode
<pre> ALTER PROCEDURE [dbo].[Update_Producto_StatusCode] -- Add the parameters for the stored procedure here @NSerie VARCHAR(100), @NuevoStatus VARCHAR(20) AS BEGIN -- SET NOCOUNT ON added to prevent extra result sets from -- interfering with SELECT statements. SET NOCOUNT ON; -- Insert statements for procedure here update Productos set TestStatusCode = @NuevoStatus where NSerie = @NSerie; END </pre>

6.4.- Desarrollo del sistema.

Como se pudo observar en los diagramas de operación, el programa se puede explicar en tres operaciones fundamentales del programa principal, cuales harán uso del controlador *service* para la realización de tareas que requieran el uso de diferentes clases. Este enfoque se aplicó de manera consistente en los distintos módulos del sistema, particularmente en las clases encargadas de la comunicación con el dispositivo *SeaLevel* y en la validación de los procedimientos almacenados de la base de datos.

6.4.1- Conexión al instrumento.

Al momento de presionar el botón para conectar los instrumentos, se debe primero asegurar que cada uno contenga su respectiva dirección del puerto. Esto es importante ya que, si el operador se equivoca al momento de designar la dirección de los instrumentos y estos resultan ser erróneos al conectarse, no podrán realizar las operaciones adecuadas. Por lo tanto, se desarrolló un sistema que al momento de realizar la conexión busca la identidad del instrumento (a partir del comando de IDN). Como se trabaja con una fuente y un multímetro, se puede buscar el modelo de cada uno y así asegurarse de que sean la dirección correcta. Si no lo son el sistema inmediatamente desconecta los dos.

Es importante considerar que este sistema se diseñó en mente con dos instrumentos completamente diferentes, si fueran el mismo, se tendría que comparar de otra manera para reconocer cual debe ser la dirección respectiva de cada uno.

En este mismo apartado requiere también de la conexión del *SeaLevel* para la comunicación con los relés, esto es dado a que como se comentó previamente, estos son los que permitirán controlar que prueba se realizara para el proyecto. Como este se reconoce usando la librería previamente mencionada *System.IO.Ports*, no existe problemas al momento de encontrar la dirección (también se facilita ya que se denomina como COM6 y es el único de su tipo).



Dado sea el caso en el que se inicializa el programa y no se encuentran direcciones a asignar un instrumento o al *SeaLevel*, es posible volver a refrescarlos usando el botón *Refresh Locations* que permite al usuario cargar otra vez las direcciones de los puertos, pero para evitar un error al momento de conectar los instrumentos, se deselectan todos los elementos del *ComboBox*.

6.4.2- Prueba del producto.

Como la estación de prueba debe realizar diferentes medidas para cada modelo, primero es importante que al momento de presionar el botón “Initiate Test”, este procederá a seleccionar todos los productos que no se hayan medido. Esto es importante porque se generará una cola que indica la cantidad de pruebas a realizarse.

Lo primero que se debe considerar es que estas pruebas deben mostrar su progreso dentro de la interfaz, en el *Test Panel*, para poder cumplir con este requisito se utilizó un hilo que permite ver como este operara entre cada prueba, asegurándonos de habilitar su propiedad de “*isBackground*” para poder interactuar con las ventanas de *TabControl*, pero es importante saber que los elementos como *Labels* o *LEDs* del programa no se pueden refrescar dentro del *loop*, inclusive existen algunas operaciones que modifican otros elementos que son intolerantes a los hilos que llegan a interrumpir el programa. Esto representó un inconveniente durante el desarrollo, ya que impedía cumplir con los objetivos del proyecto, la solución a esto fue utilizar un *invoke*, esto llega ser esencial para el desarrollo de este programa. Adelante se puede observar un fragmento que utiliza *invoke*.

```
this.Invoke((MethodInvoker)() =>
{
    R_Success.Value = true; meter2.Value = result;
    LB_R_MEAS.Text = "Voltage: " + result.ToString() + " V";
});
```

Es importante reconocer que esto solo debe usarse solo cuando se desea actualizar dichos elementos y no encapsular toda la operación del hilo, si se realiza esto genera un problema en el



cual se actualiza parte de los elementos en ciertos casos no se actualiza hasta que vuelva a ejecutarse el *invoke* o simplemente solo se ve parcialmente el cambio. Un ejemplo para entender mejor sería suponer que actualizamos un *label* para que indique el resultado del voltaje de una prueba es 12.1945V, pero al momento de que se ejecute solo se observa parcialmente la primera decima.

Es importante destacar que el hilo siempre se ejecuta, pero para evitar que este realice lecturas sin aprobación del operador, se implementó una bandera que checa si se presionó el botón para realizar la prueba del producto siguiente en la cola. Cada producto tiene que realizar 4 pruebas hasta que todas sean exitosas o una marque un error, si lo último llega a suceder, todas las demás pruebas para ese producto serán canceladas y anotadas con un NT (*Not Tested*) pero la que fallo será marcada con un “*FAIL*”. Para el caso del fallo es sencillo programar, ya que para cada elemento *TestPlan* se debe crear un *TestPlan Result* que será registrado en el reporte y la base de datos. Pero si falla, se tiene que realizar casos en donde dependiendo en que parte de la prueba fallo, el programa debe registrar las demás pruebas que no se probaran y marcarlas como se mencionó previamente con un NT el cual. Un ejemplo es que, si falla en la prueba de continuidad, este debe registrar los siguientes 3 *TestPlan* como no probados. El siguiente fragmento muestra el cómo se realiza.

case 3:

```
for (int i = 0; i < 3; i++)  
{ reg(ns, i + 1, tpID + i + 1); }  
NetTP_RId += 4;  
x = 4;  
break;
```

En donde el ciclo registra las veces necesarias NT y lo sube en la base de datos (el método *reg* es el que se encarga de registrar estos elementos al reporte y base de datos).



Pero para poder evaluar si paso o no las pruebas se requiere uso de las clases instrumentos, específicamente el multímetro y fuente para poder diseñar las pruebas a realizar para el circuito y analizar si cumplen con los límites de los *Test Plan*, dependiendo de ello, su resultado (es decir el *Test Result*) cambiara el valor de su estatus.

Es importante considerar también que solo pueden existir un producto único, lo que nos permite distinguirlos son el número de serie que contiene cada uno de estos. Si se inserta un producto a la cola, primero se debe reconocer si ese producto no se encuentra dentro de la tabla de *TestResult*, es decir, el sistema analiza si paso por una prueba o no. Si resulta que ya se probó ese modelo, no se puede ingresar a la cola para realizar pruebas.

6.4.3- Búsqueda.

La interacción entre la aplicación en C# y la base de datos SQL Server se implementó mediante la clase *ProductRepository*, responsable de gestionar el acceso a datos. Dicha clase ejecuta operaciones de lectura y escritura a través de procedimientos almacenados, garantizando una comunicación estructurada, segura y eficiente con la base de datos. Para la obtención de los reportes, se utilizó el objeto *DataTable* cual permite transportar la información para posteriormente cargarlo a los DGV (*DataGridView*) en donde se desea presentar la información.

Para filtrar los diferentes resultados de la base de datos, se requiere de un *ComboBox* que decide que en base a que parámetro se desea filtrar (número de serie, rango de fechas, modelo, estatus), donde la interfaz te permitirá modificar los valores. El reporte solo se puede mostrar cuando se presiona el botón de búsqueda, el cual a partir del procedimiento almacenado de *SP_Filter* encontrará y retornará la información encontrada.



Para poder realizar este filtro en un solo SP, se requiere de una identificación para que SQL reconozca que se debe filtrar, para esto se utilizó una variable denominada *@OperationNumber*, el siguiente fragmento de código muestra el valor que debe cumplir para que se filtren los números de serie.

```
----- Búsqueda de Numero de Serie  
if @OperationNumber = 1  
Begin  
Select * From TestResult as r  
where r.NSerie like '%' + @NS + '%'  
End
```

Como se puede observar para el número de serie, los comandos de porcentaje (o comodines) permitirán buscar para cualquier número de serie que contenga los valores de *@NS*. Dependiendo en base que otro dato se desea filtrar el valor de *@OperationNumber* cambiara.

7.- Limitaciones.

- El sistema estará limitado a un entorno local, sin conexión a bases de datos externas ni almacenamiento en la nube.
- El sistema depende de que los instrumentos utilizados sean compatibles con VISA y SCPI.
- El sistema no contempla comunicación remota ni ejecución distribuida de pruebas.
- La cola se manejará en memoria, por lo que la información se perderá al cerrar la aplicación.

8.-Resultados

Register Queue Test Panel History Search

Multimeter Port Location: Source Port Location: SeaLevel Port:

GPIB1::22::INSTR

GPIB1::3::INSTR

COM6

Connect Instrument

Refresh Locations

Disconnect Instrument

Register Data

Instruments Connected Successfully

OK

Fig. 8.1 -Resultados de la Register

En la Figura 8.1 se muestra la interfaz gráfica del sistema desarrollado, correspondiente a la etapa de conexión y configuración de instrumentos. En esta fase, el software detectó correctamente los recursos disponibles mediante la interfaz VISA, identificando el multímetro digital y la fuente de alimentación conectados a través del bus GPIB, así como el puerto COM asignado al módulo de control SeaLevel.

Register	Queue	Test Panel	History	Search	
ID TP	Test Number	Version	Test Name	LI	LS
5	101	1	Continuity Test	0	10
6	102	1	Voltage Test	11	12.5
7	103	1	CCW Test	-11.75	-10.7
8	104	1	Motor Vel Test	3.8125	4.56
9	201	2	Continuity Test	0	2
10	202	2	Voltage Test	5.635	6.46
11	203	2	CCW Test	-8.75	-7.75
12	204	2	Motor Vel Test	10.325	11.4
13	201	2	Continuityv Test	0	2

Fig. 8.2- Resultados de queue.

En la Figura 8.2 se presenta la sección *Test Panel* del sistema desarrollado, donde se muestra la lista de planes de prueba (*Test Plans*) cargados correctamente en la cola de ejecución. Cada registro corresponde a una prueba específica asociada a un identificador único (*ID TP*), un número de prueba (*Test Number*) y una versión del plan (*Version*), lo que permite un control estructurado y ordenado de las pruebas a realizar.

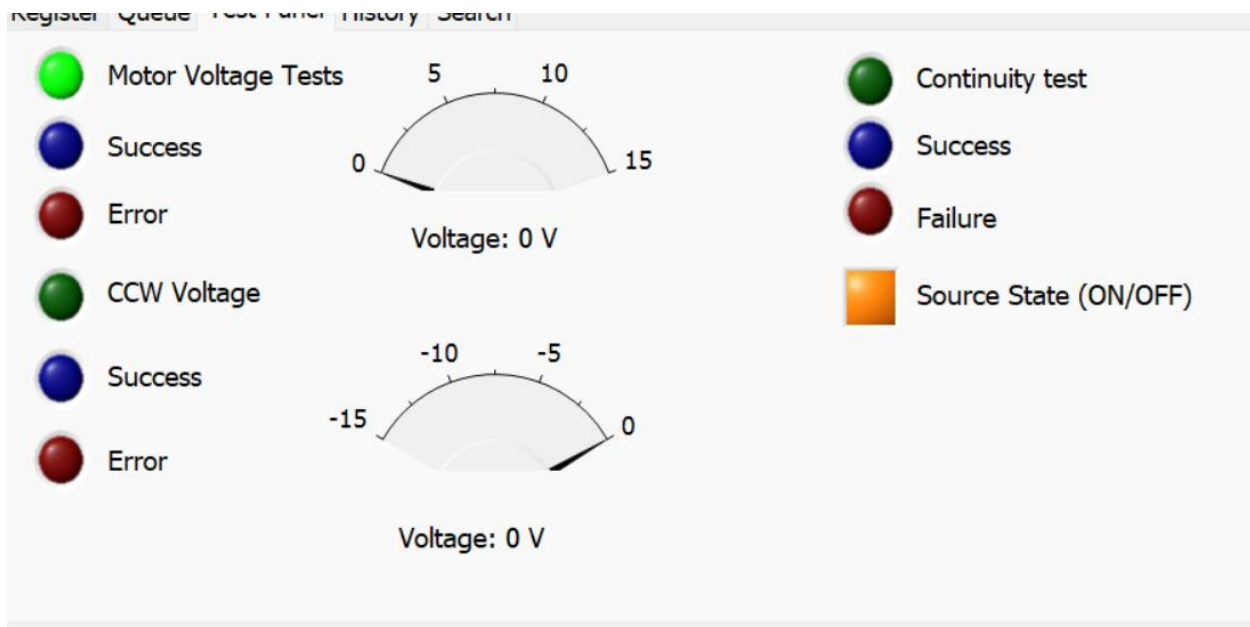


Fig. 8.3- Resultados de los test plans.

En la Figura 8.3 se muestra el Test Panel del sistema, que ve en tiempo real el estado de las pruebas eléctricas ejecutadas sobre el dispositivo bajo prueba. La interfaz integra indicadores luminosos y medidores analógicos que facilitan la interpretación inmediata del comportamiento del sistema durante cada etapa de medición.

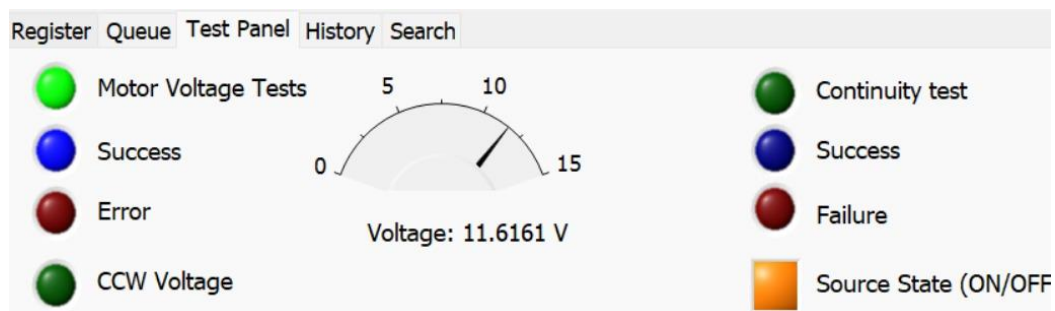


Fig. 8.4- Resultados de los test plans.

Para la prueba Motor Voltage Tests, el sistema cuenta con un medidor que representa el nivel de voltaje aplicado al motor, así como indicadores visuales de estado que señalan el resultado de la prueba. El color verde indica la habilitación o ejecución de la prueba, mientras que los indicadores de Success y Error permiten identificar de forma clara si el valor medido se encuentra dentro de los límites establecidos.

	Test Result Id	Series Number	Test Plan Id	Test Result Value	Status Code	Test Date Time
▶	1	SN0001A	1	2.9938	Passed	12/15/2...
	2	SN0001A	2	11.4712	Passed	12/15/2...
	3	SN0001A	3	-10.4365	Failed	12/15/2...
	4	SN0001A	4	0	NT	12/15/2...
*						

Fig. 8.5- Resultados de los test plans.

Se muestra la interfaz correspondiente al apartado de resultados de pruebas del sistema. En esta sección se visualiza una tabla de resultados, en la cual se registran de manera estructurada los obtenidos tras la ejecución de los planes de prueba.

La tabla incluye campos como *Test Result ID*, *Series Numbre*, *Test Plan ID*, *Test Result Value*, *Status Code* Y *Test Date Time*. Estos Campos Permiten identificar cada resultado de prueba, asociarlo a un número obtenido, el estado de la prueba y la fecha y hora en que fue realizada.

Los resultados muestran diferentes estados, tales como *Passed*, *Failed*, y *NT (Not Tested)*, que señala pruebas pendientes o no realizadas. Los valores numéricos registrados en la columna *Test Result Value* correspondiente a las mediciones obtenidas durante las pruebas.

En la parte superior se mantiene el botón *Display Test Plans*, que permite actualizar o consultar los planes de pruebas asociados, mientras que la parte inferior se presenta una barra de estado que muestra información general del sistema, como el estado de la cola de pruebas y la hora del sistema.

Register Queue Test Panel History Search

Search Parameter

Serial Number
Model
Serial Number
Status Code

Search

A
SN9999A
PASS

Dates

Wednesday, December 17, 2025
Wednesday, December 17, 2025

	TestResult	NSerie	TestPlanId	TestResult	StatusCode	TestDateTi
▶	9	SN9999A	1	1.000000	PASS	12/16/2...
	10	SN9999A	2	5.800000	FAIL	12/16/2...
	11	SN9999A	3	0.000000	NT	12/16/2...
	12	SN9999A	4	0.000000	NT	12/16/2...
*						

Fig. 8.6- Búsqueda filtrada en base a número de serie.

Search Parameter

Status Model Serial Number Status Code Dates

	TestResult\	NSerie	TestPlanId	TestResult\	StatusCode	TestDateTi
▶	2	SN0001A	2	5.800000	FAIL	12/16/2...
	6	SN0003A	2	5.800000	FAIL	12/16/2...
	10	SN9999A	2	5.800000	FAIL	12/16/2...
*						

Fig. 8.7- Búsqueda filtrada en base a estatus.

El sistema de filtros resulta eficiente, ya que permite localizar con facilidad los resultados de las pruebas requeridas. Gracias a este mecanismo, el operador puede consultar la información correspondiente a distintos escenarios de manera rápida y sencilla, optimizando el acceso y análisis de los datos.

9.- Conclusión.

El proyecto cumple con la mayoría de los objetivos planteados, ya que permite probar y analizar distintos modelos, determinando si cumplen o no con los requisitos necesarios para aprobar la prueba. Este proyecto representa la culminación de los conocimientos adquiridos durante el semestre, aplicados al desarrollo de un proceso de pruebas para que no requiera de la interacción del operador y evite posibles daños en la placa durante el desarrollo de las pruebas.

Esto es importante para nuestra formación como ingenieros, ya que proporciona el conocimiento para el desarrollo de habilidades y pensamiento crítico para el diseño, implementación y validación de sistemas de prueba.

10- Bibliografías

Braga, N. C. (s. f.). *Control de Velocidad para Motores CC (ART139S)*. INCB.

<https://newtoncbraga.com.mx/articulos/9-articulos-tecnicos-y-proyectos/823-control-de-velocidad-para-motores-cc-art139s>

Securing Modbus Communications: Modbus TLS and Beyond. (s. f.). Sealevel Systems, Inc.

<https://www.sealevel.com/securing-modbus-communications-modbus-tls-and-beyond>

Sistemas de pruebas y medidas, una división de Emerson. (2022, 2 septiembre). NI.

https://www.ni.com/es.html?srsId=AfmBOorkouy-59Py6O9v_8fvMiLDOXXD_vmeLT55Z9aV6c9zUeqN3E5z



Steven. (s. f.). *inversion de giro en motores*. Scribd.

<https://www.scribd.com/document/793645332/inversion-de-giro-en-motores>

Información General sobre NI-VISA. (2006, 31 agosto). NI.

<https://www.ni.com/es/support/documentation/supplemental/06/ni-virtual-instrument-software-architecture.html?srsId=AfmBOoq5-FRQ4VF7qsBVfyfRSvicLTbHRe-QCvkQjC5-ZqH-g8IhAWII>